

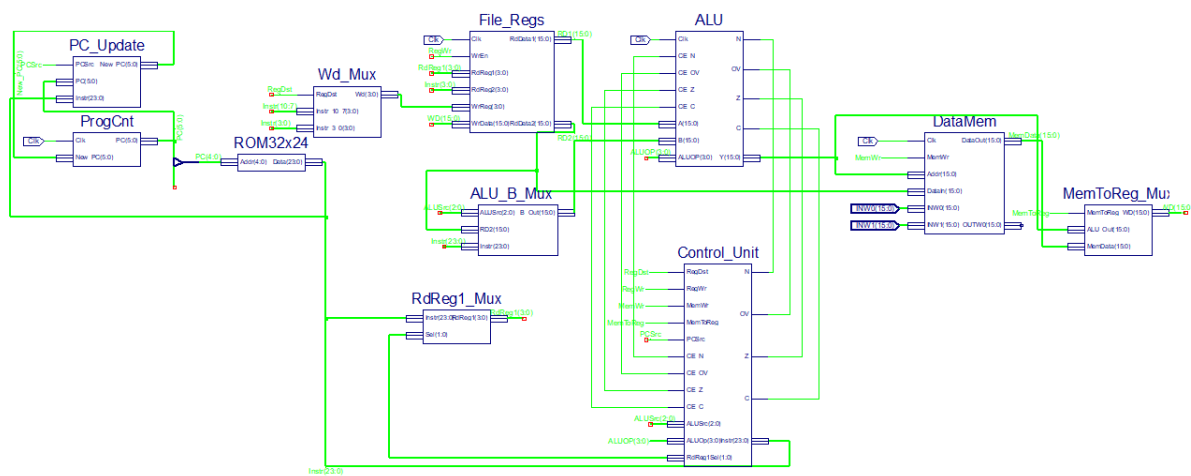
Proiect PIC24

Proiect #71- Stan Marian Gabriel

Proiectul 71

Non-jump Instructions	Flags	Jump Instructions
ASR Wb,#lit4,Wnd	OV	BRA OV,Expr
SUB Wb,#lit5,Wd	C	BRA C,Expr
COM Ws,Wd	N	BRA N,Expr
AND #lit10,Wn	Z	BRA Z,Expr

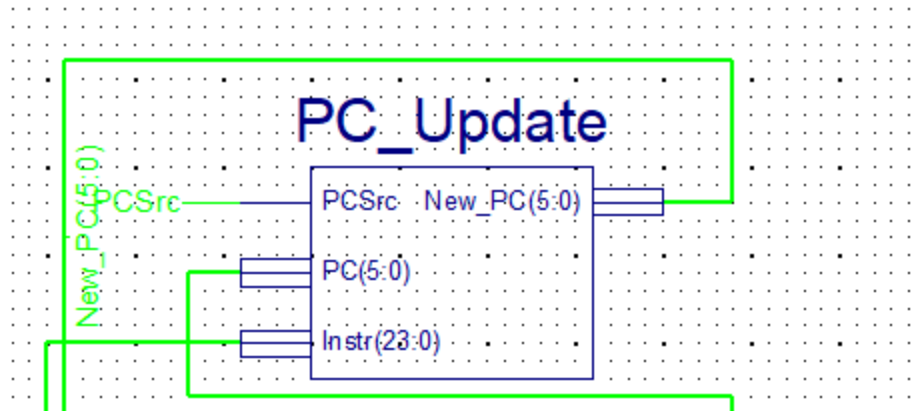
Schema bloc a procesorului PIC24:



Rolul fiecarui bloc din schema:

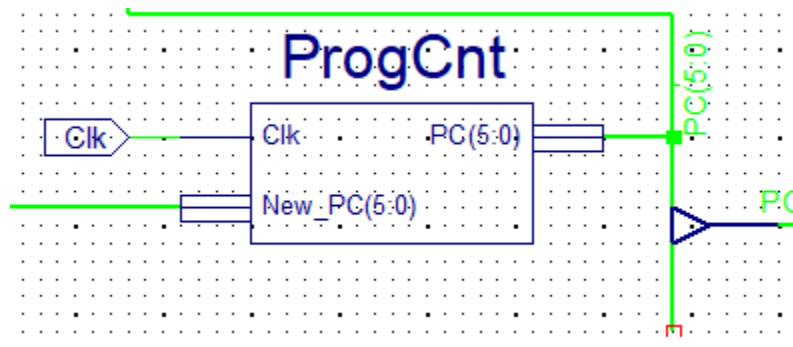
1) PC_Update

Primește valoarea curentă a PC, semnalul PCSrc de la Control_Unit și offsetul din instrucțiune (Instr(5:0)). Dacă PCSrc=1 înseamnă că facem salt, deci $New_PC = PC + 1 + \text{offset}$. Dacă nu, $New_PC = PC + 1$ (execuție normală, secvențială).



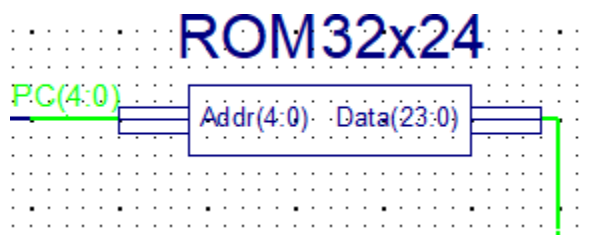
2) ProgCnt

Este registrul PC. Pe frontul crescator de ceas retine valoarea New_PC primita de la PC_Update.



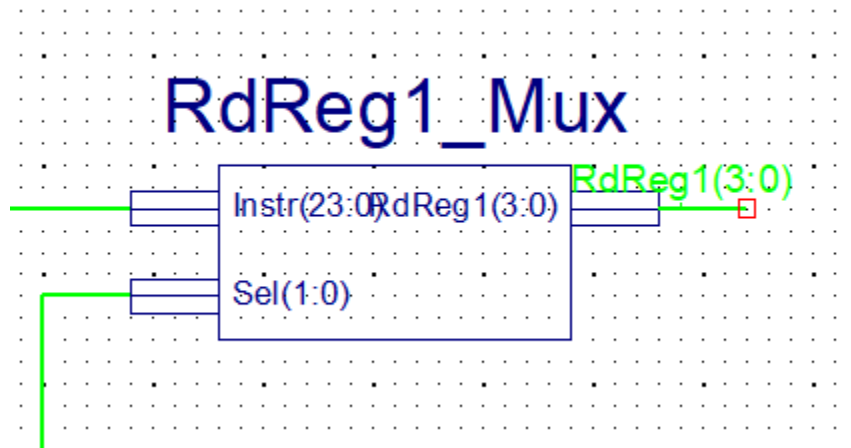
3) ROM32x24

Este memoria de program, 32 de locatii de cate 24 de biti. Adresa vine din PC (5 biti), iesirea e instructiunea curenta, Instr(23:0).



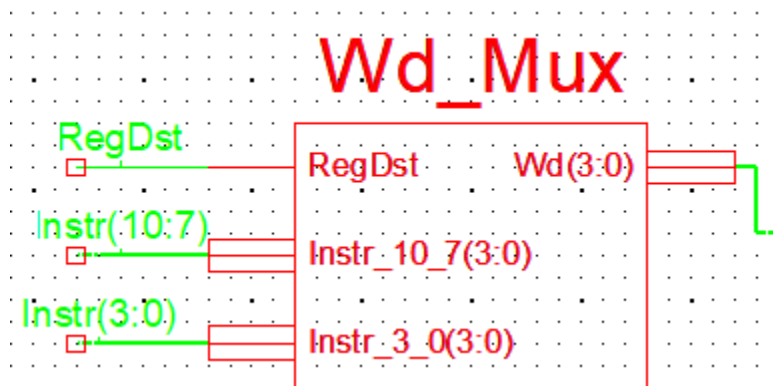
4) RdReg1_Mux

La unele instructiuni (ADD, SUB, AND, IOR) registrul Wb e pe biti 18:15, dar la ASR e pe biti 14:11, iar la COM si AND cu literal operandul e pe biti 3:0. Ca sa nu facem un fir separat pentru fiecare caz, avem mux-ul asta care alege bitii corecti in functie de RdReg1Sel, semnal trimis de Control_Unit.



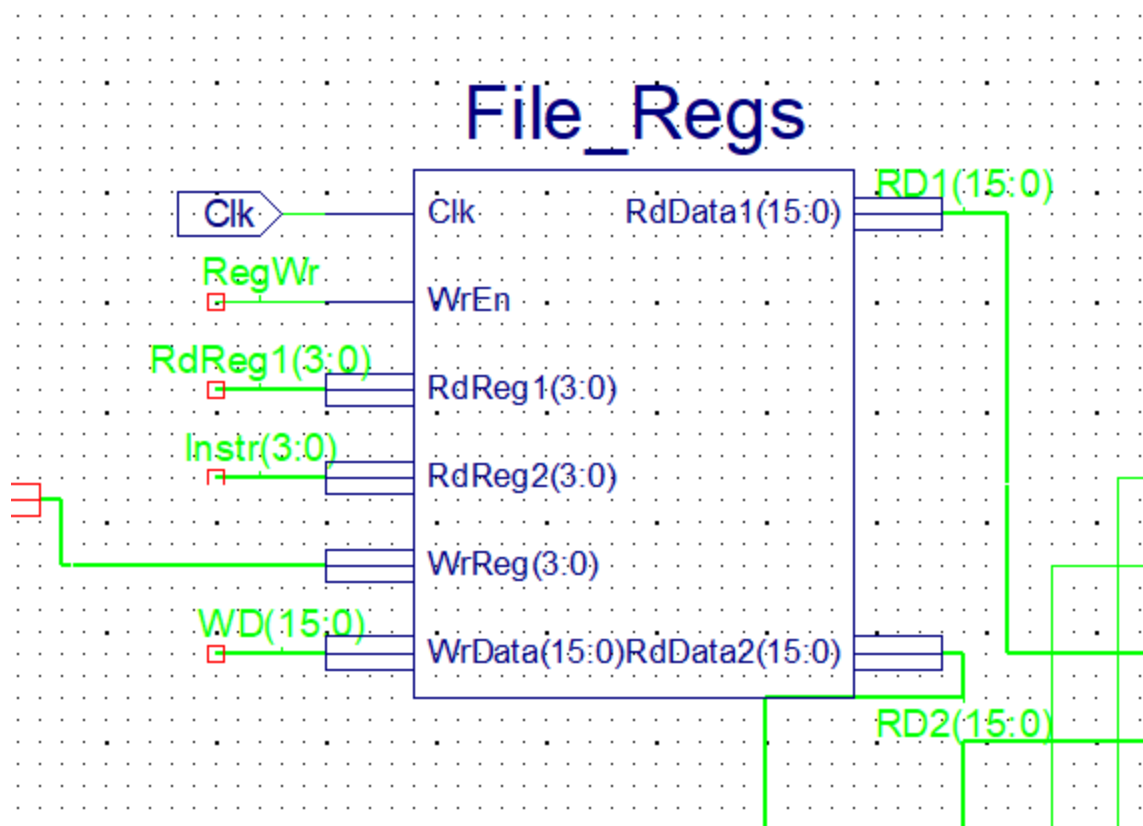
5) Wd_Mux

Alege intre Instr(10:7) si Instr(3:0) ca sa stabileasca ce registru e destinatie, in functie de RegDst. La ADD/SUB/AND/IOR destinatia e pe 10:7, dar la MOV f,Wnd si AND cu literal e pe 3:0.



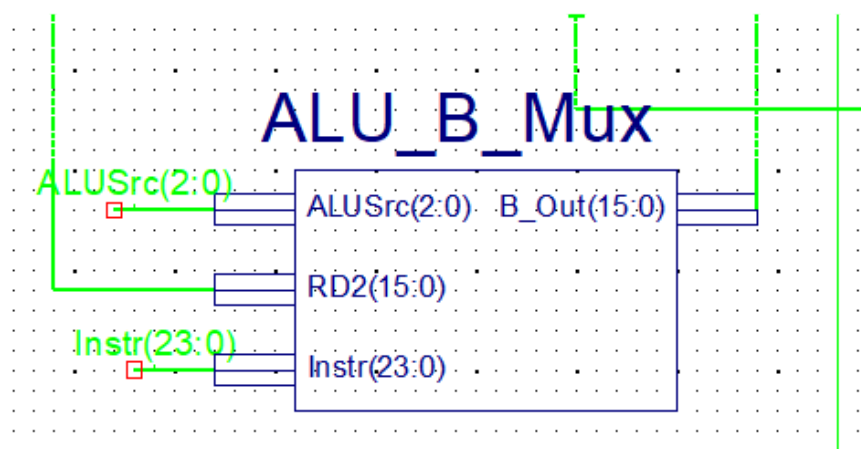
6) File_Regs

Cele 16 registre W0-W15. Poate citi doua registre deodata (pentru instructiunile care au nevoie de 2 operanzi) dar scrie doar intr-unul singur, pe frontul de ceas, cand WrEn=1.



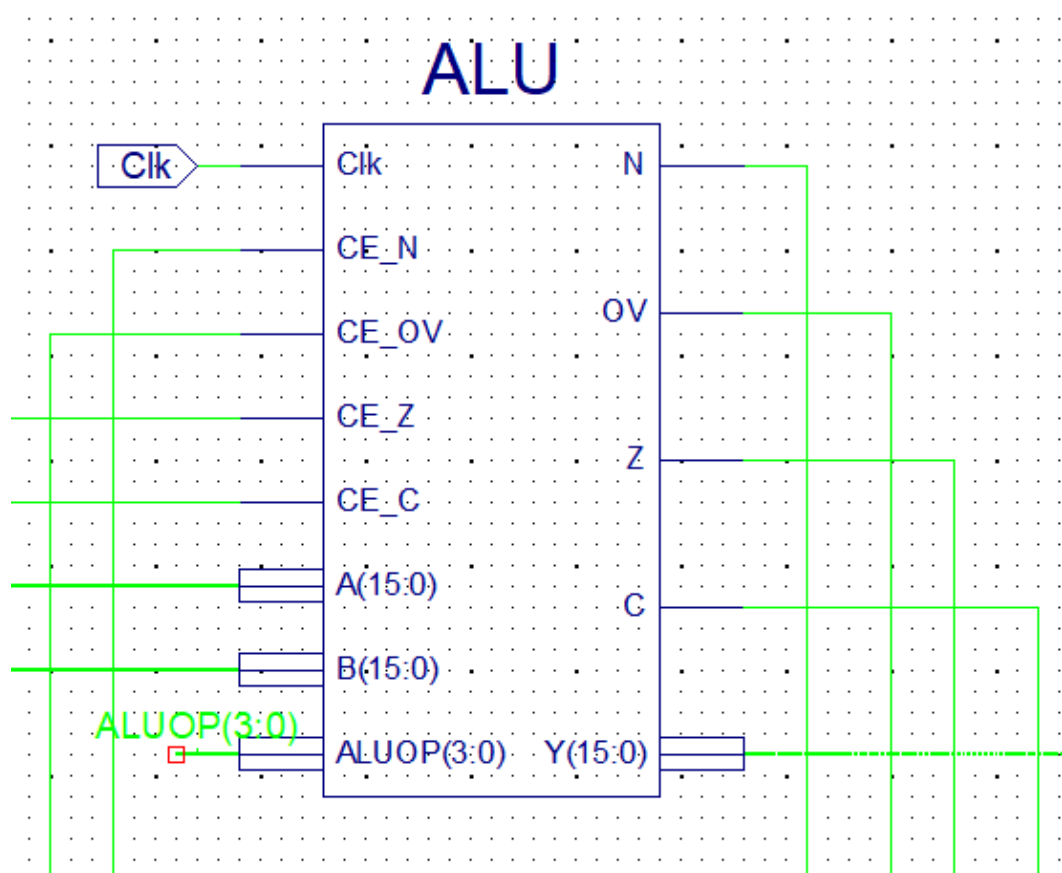
7) ALU_B_Mux

Alege al doilea operand al ALU: fie registrul Ws citit din File_Regs, fie un literal luat direct din instructiune (diferit ca pozitie pentru ASR, SUB cu literal si AND cu literal), fie adresa pentru instructiunile MOV cu memorie.



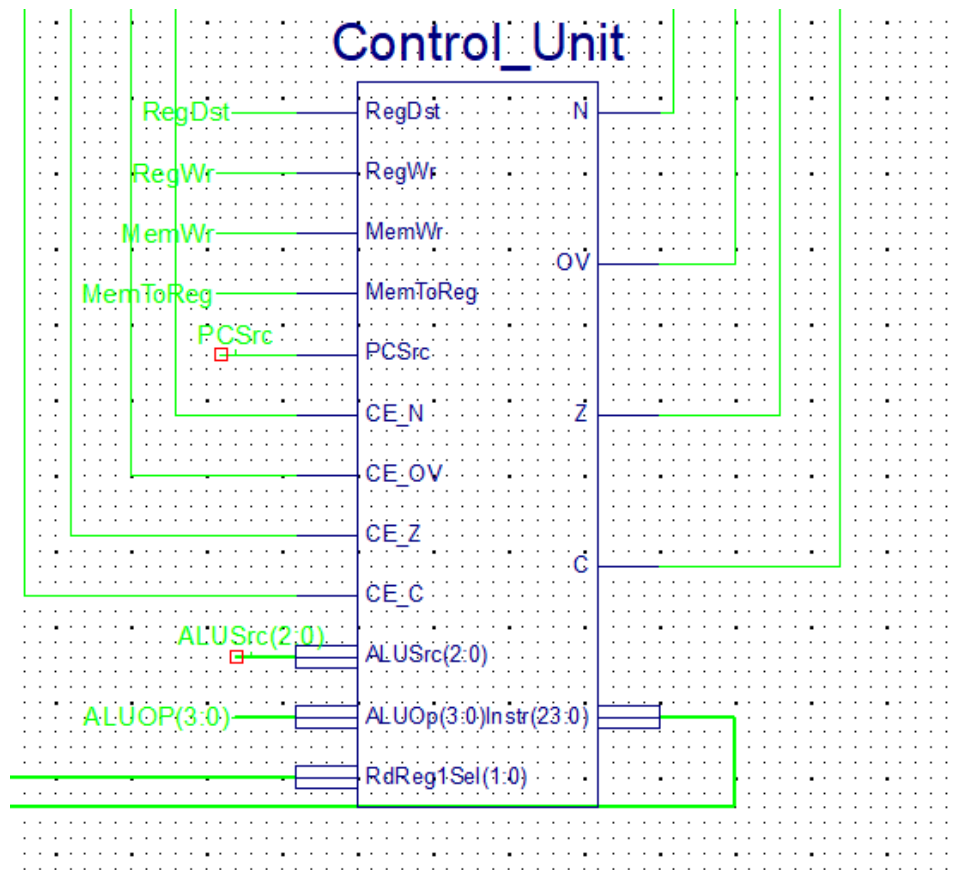
8) ALU

Face operatia ceruta de ALUOP: ADD, SUB, AND, IOR, ASR sau COM. Tot aici se calculeaza si se memoreaza flagurile N, OV, Z, C, fiecare cu propriul semnal de Clock Enable, ca sa nu se schimbe flagul cand instructiunea curenta nu-l afecteaza.



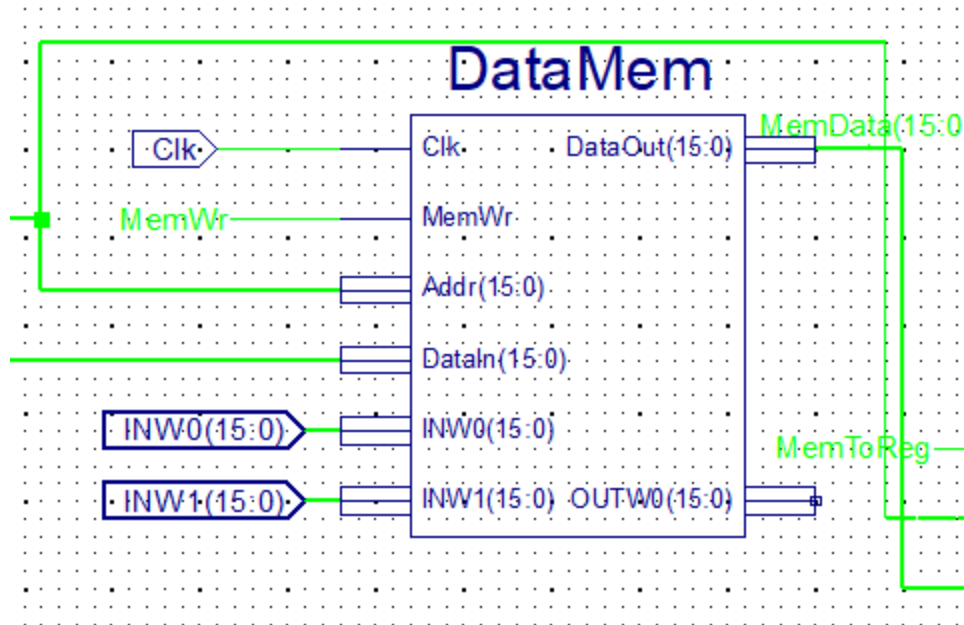
9) Control_Unit

Decodifica instructiunea si scoate toate semnalele de comanda: RegDst, RegWr, ALUSrc, ALUOp, MemWr, MemToReg, PCSrc, RdReg1Sel si cele 4 Clock Enable -uri pentru flaguri. La saltul conditionat, PCSrc ia direct valoarea flagului cerut (N, OV, Z sau C).



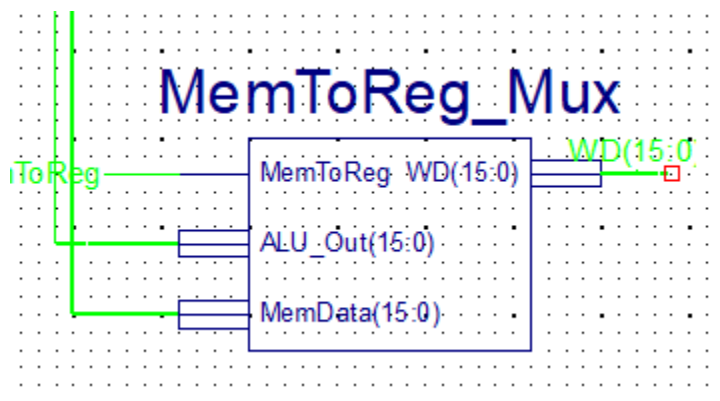
10) DataMem

Memoria de date, 16 locatii de 16 biti (adresele 0x1000-0x101F) plus INW0 (0x1020) si INW1 (0x1022) care sunt doar de citire, si OUTW0 (0x1024) care e doar de scriere. Adresa vine direct din instructiune, nu din ALU (adresare directa).



11) MemToReg_Mux

Alege ce se scrie in File_Regs: iesirea ALU sau iesirea din DataMem, in functie de MemToReg. Se foloseste doar la MOV f,Wnd.



Rolul fiecarui semnal trimis de Unitatea de Control:

RegDst

Alege intre Instr(10:7) si Instr(3:0) ca adresa a registrului destinatie, in Wd_Mux. E 0 la ADD/SUB/AND/IOR/ASR/COM, si 1 la MOV f,Wnd si AND #lit10 (unde destinatia e pe alti biti).

RegWr

Decide daca instructiunea curenta scrie sau nu intr-un registru din File_Regs. E 0 la BRA si la MOV Wns,f (acestea nu scriu in registre).

ALUSrc

Decide ce ajunge pe intrarea B a ALU: registrul Ws, sau unul din cei trei literali diferiti (ASR, SUB lit5, AND lit10), sau adresa pentru MOV cu memorie.

ALUOp

Transmite ALU-ului ce operatie sa faca: ADD, SUB, AND, IOR, ASR, COM sau pass-through (folosit la MOV, unde de fapt rezultatul nu conteaza, pentru ca se scrie direct din memorie).

MemWr

Activeaza scrierea in DataMem. Singura instructiune care scrie in memorie e MOV Wns,f.

MemToReg

Decide ce se scrie in File_Regs: iesirea ALU sau iesirea DataMem. E 1 doar la MOV f,Wnd (cand citim din memorie intr-un registru).

PCSrc

Trasnmite catre PC_Update daca trebuie facut saltul sau nu. La BRA necoditionat e mereu 1. La BRA conditionat ia direct valoarea flagului cerut (Z, N, C sau OV). La restul instructiunilor e 0.

CE_N, CE_OV, CE_Z, CE_C

Cate un semnal de Clock Enable pentru fiecare flag din ALU. Daca o instructiune nu afecteaza un anumit flag (de exemplu AND nu afecteaza C si OV), semnalul de CE corespunzator ramane 0, iar flagul isi pastreaza valoarea veche.

RdReg1Sel

Decide ce biti din instructiune reprezinta registrul sursa citit pe RdReg1, pentru ca pozitia difera intre formate (18:15 la ADD/SUB/AND/IOR, 14:11 la ASR, 3:0 la COM si AND cu literal).

Tabelul de adevar al Unitatii de Control:

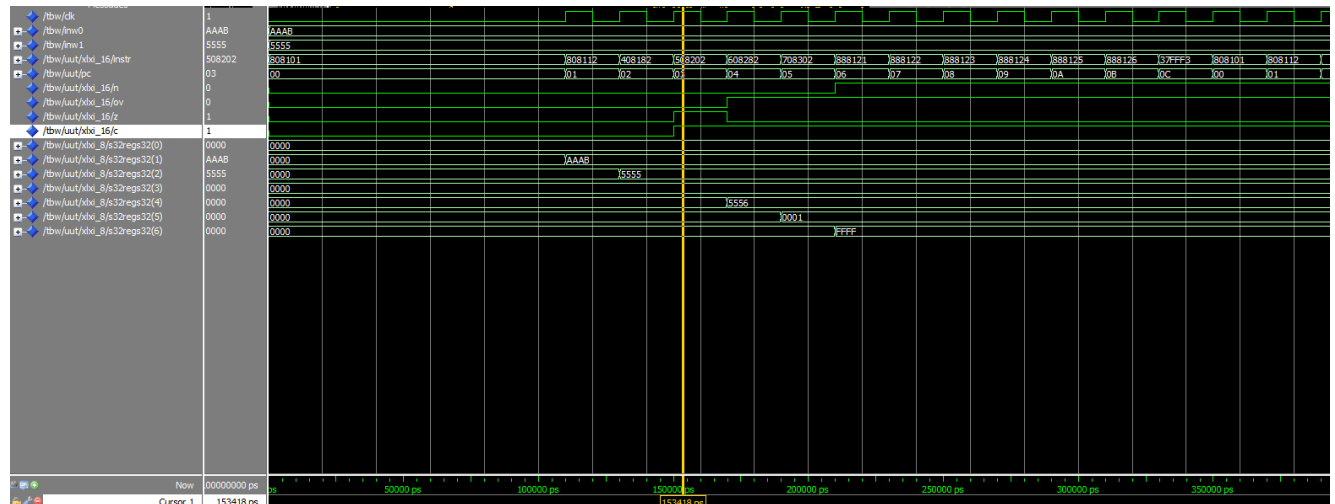
Instructiune	Opcode	RegDst	RegWr	ALUSrc	ALUOp	MemWr	MemToReg	PCSrc	CE_N	CE_OV	CE_Z	CE_C	RdReg1Sel
ADD Wb, Ws, Wd	01000	0	1	000	0000	0	0	0	1	1	1	1	00
SUB Wb, Ws, Wd	01010	0	1	000	0001	0	0	0	1	1	1	1	00
SUB Wb, #lit5, Wd	01010	0	1	010	0001	0	0	0	1	1	1	1	00
AND Wb, Ws, Wd	01100	0	1	000	0010	0	0	0	1	0	1	0	00
IOR Wb, Ws, Wd	01110	0	1	000	0011	0	0	0	1	0	1	0	00
MOV f, Wnd	10000	1	1	100	0111	0	1	0	0	0	0	0	00
MOV Wns, f	10001	0	0	100	0111	1	0	0	0	0	0	0	00
BRA expr	x"37"	0	0	000	0111	0	0	1	0	0	0	0	00
BRA Z, expr	x"32"	0	0	000	0111	0	0	Z	0	0	0	0	00
BRA C, expr	x"31"	0	0	000	0111	0	0	C	0	0	0	0	00
BRA N, expr	x"33"	0	0	000	0111	0	0	N	0	0	0	0	00
BRA OV, expr	x"30"	0	0	000	0111	0	0	OV	0	0	0	0	00
ASR Wb, #lit4, Wnd	x"DE"	0	1	001	0100	0	0	0	1	0	1	0	01
COM Ws, Wd	x"EA"	0	1	000	0101	0	0	0	1	0	1	0	10
AND #lit10, Wn	x"B2"	1	1	011	0010	0	0	0	1	0	1	0	11

Tabelul codificarilor instructiunilor comune si specifice:

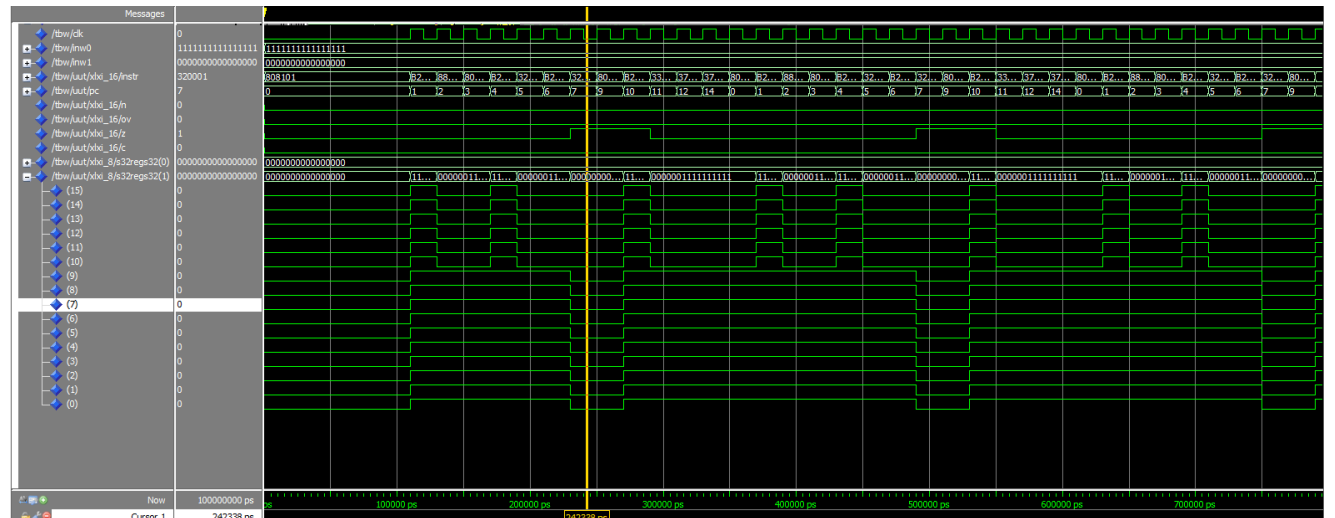
Encoding	2222 3210	1111 9876	1111 5432	11 1098	7654	3210	Flags
ADD	0100	0www	wBqq	qddd	dppp	ssss	N, OV, Z, C
SUB	0101	0www	wBqq	qddd	dppp	ssss	N, OV, Z, C
AND	0110	0www	wBqq	qddd	dppp	ssss	N, -, Z, -
IOR	0111	0www	wBqq	qddd	dppp	ssss	N, -, Z, -
MOV f, wnd	1000	0fff	ffff	ffff	ffff	dddd	none
MOV wns, f	1000	1fff	ffff	ffff	ffff	ssss	none
BRA expr	0011	0111	nnnn	nnnn	nnnn	nnnn	none
BRA C, expr	0011	0001	nnnn	nnnn	nnnn	nnnn	none
BRA N, expr	0011	0011	nnnn	nnnn	nnnn	nnnn	none
BRA OV, expr	0011	0000	nnnn	nnnn	nnnn	nnnn	none
BRA Z, expr	0011	0010	nnnn	nnnn	nnnn	nnnn	none
ASR Wb, #lit4, Wnd	1101	1110	1www	wddd	d100	kkkk	N, -, Z, -
SUB, Wb, #lit5, Wd	0101	0www	wBqq	qddd	d11k	kkkk	N, OV, Z, C
COM, Ws, Wd	1110	1010	1Bqq	qddd	dppp	ssss	N, -, Z, -
AND, #lit10, Wn	1011	0010	0Bkk	kkkk	kkkk	dddd	N, -, Z, -

REZULTATELE SIMULARII:

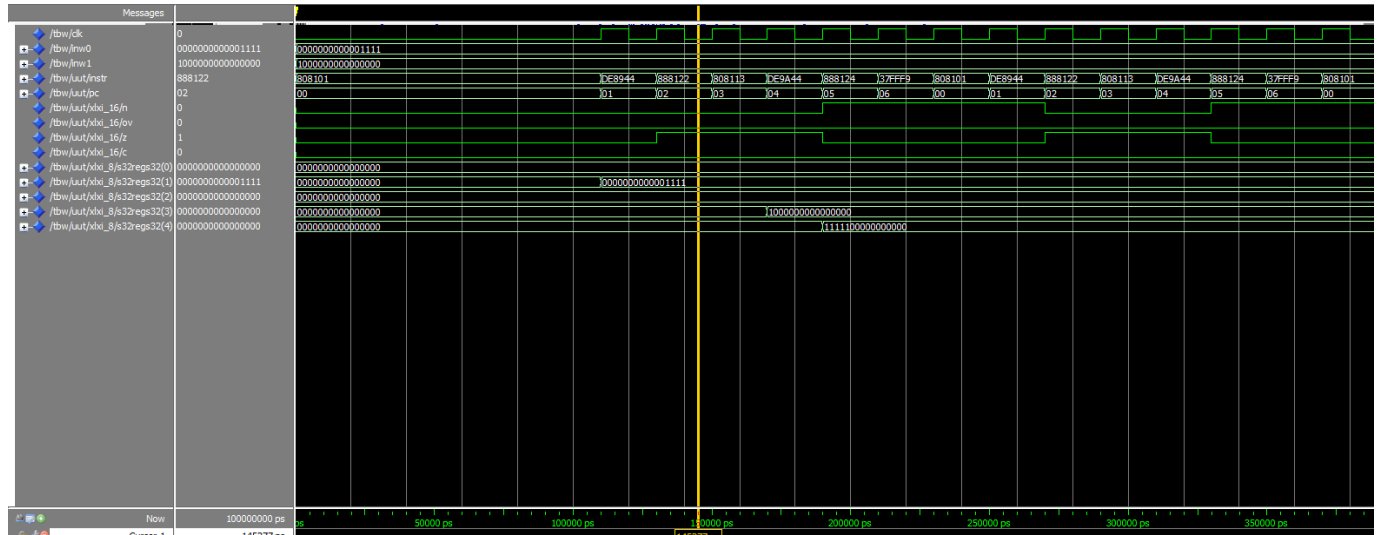
1. Secventa A (all):



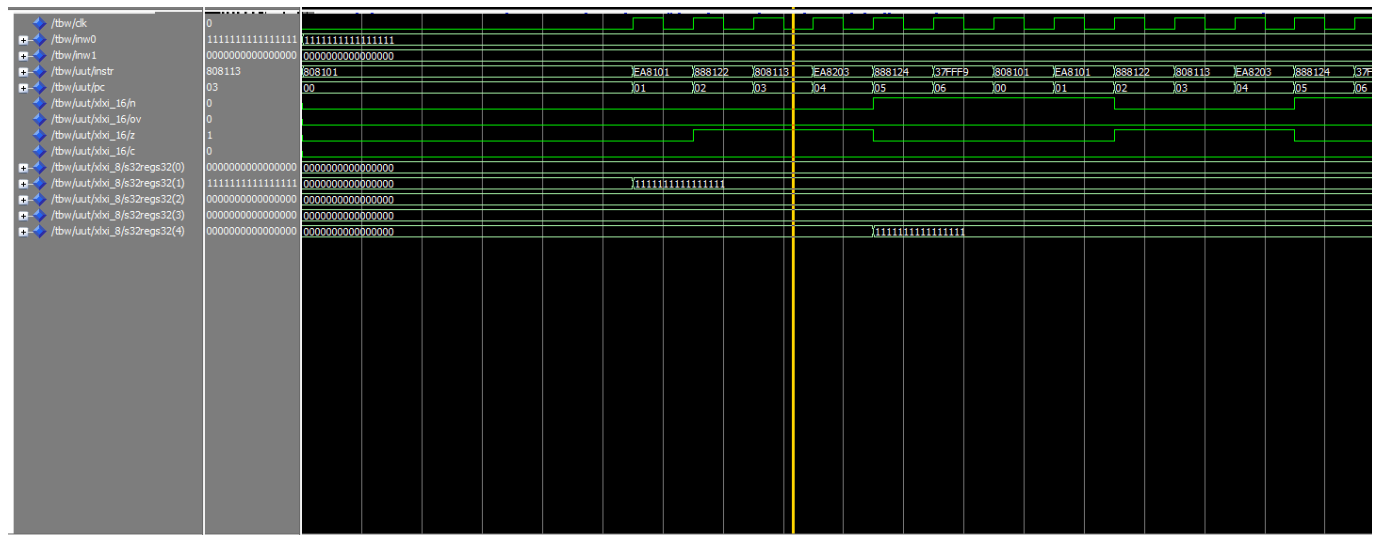
2. Secventa AND, #lit10, Wn:



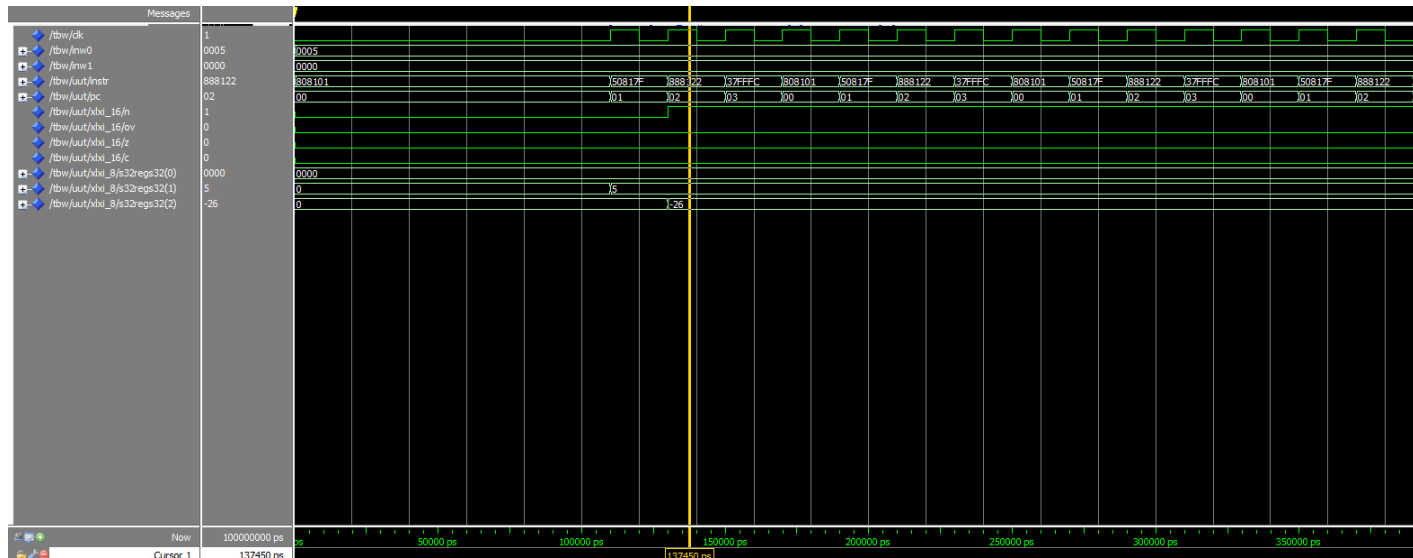
3. SECVENTA ASR Wb, #lit4, Wnd:



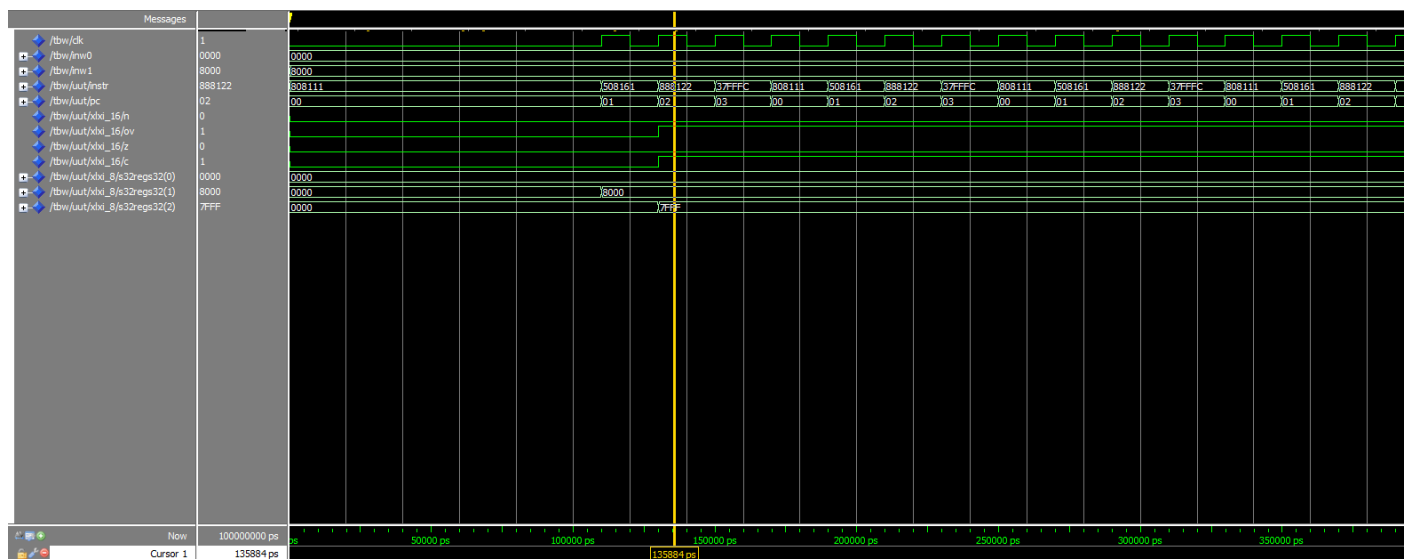
4. SECVENTA COM Ws, Wd:



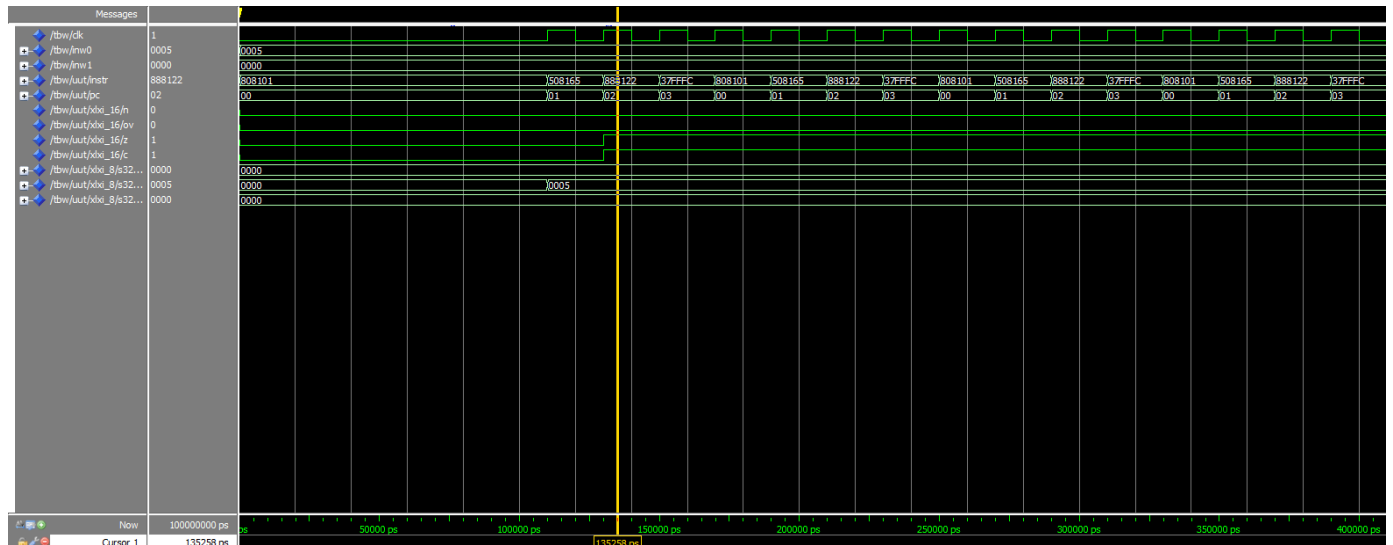
5. SECVENTA SUB, Wb, #lit5, Wd (FLAG N):



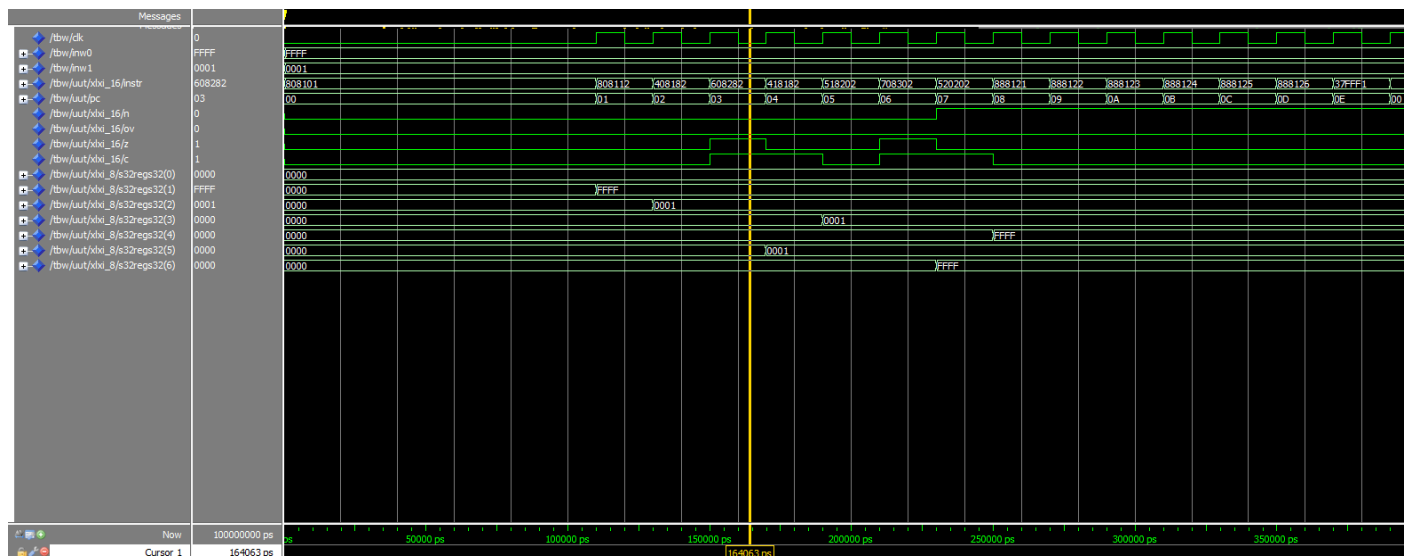
6. SECVENTA SUB, Wb, #lit5, Wd (FLAGS OV, C):



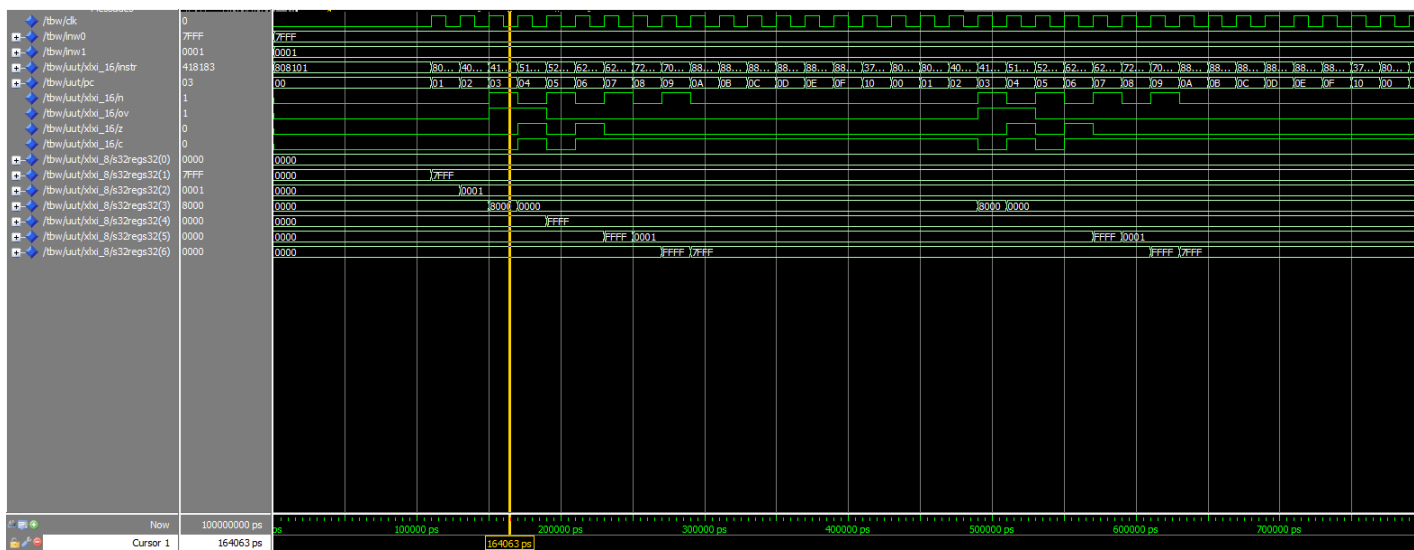
7. SECVENTA SUB, Wb, #lit5, Wd (FLAG Z , C):



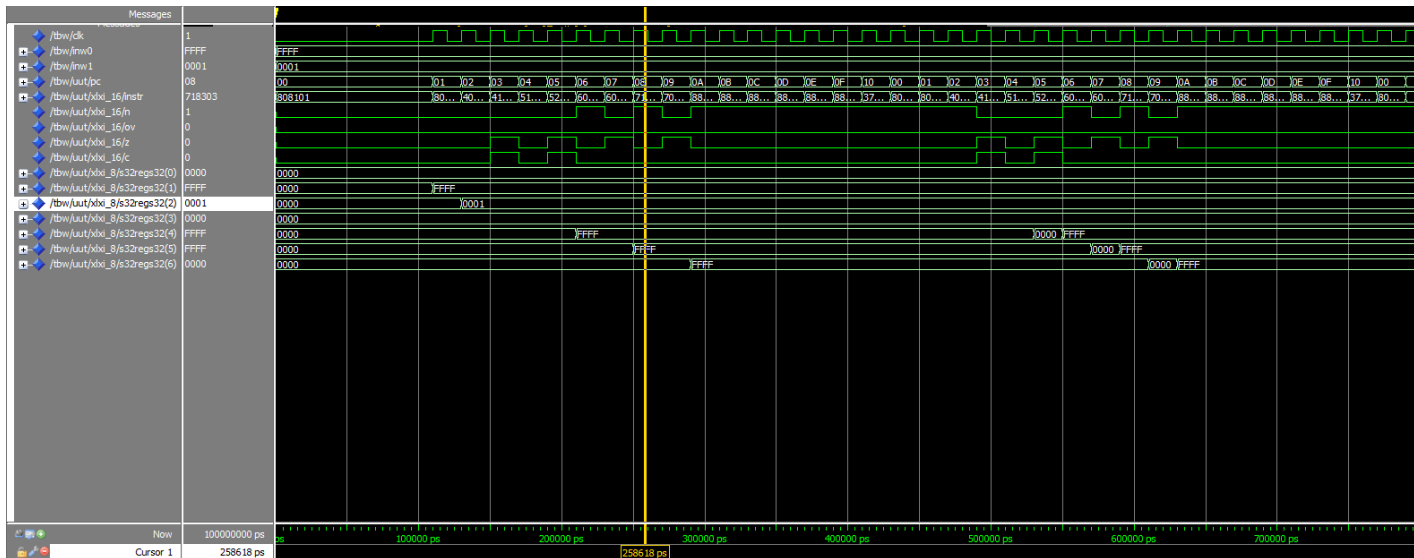
8. SECVENTA FLAG C:



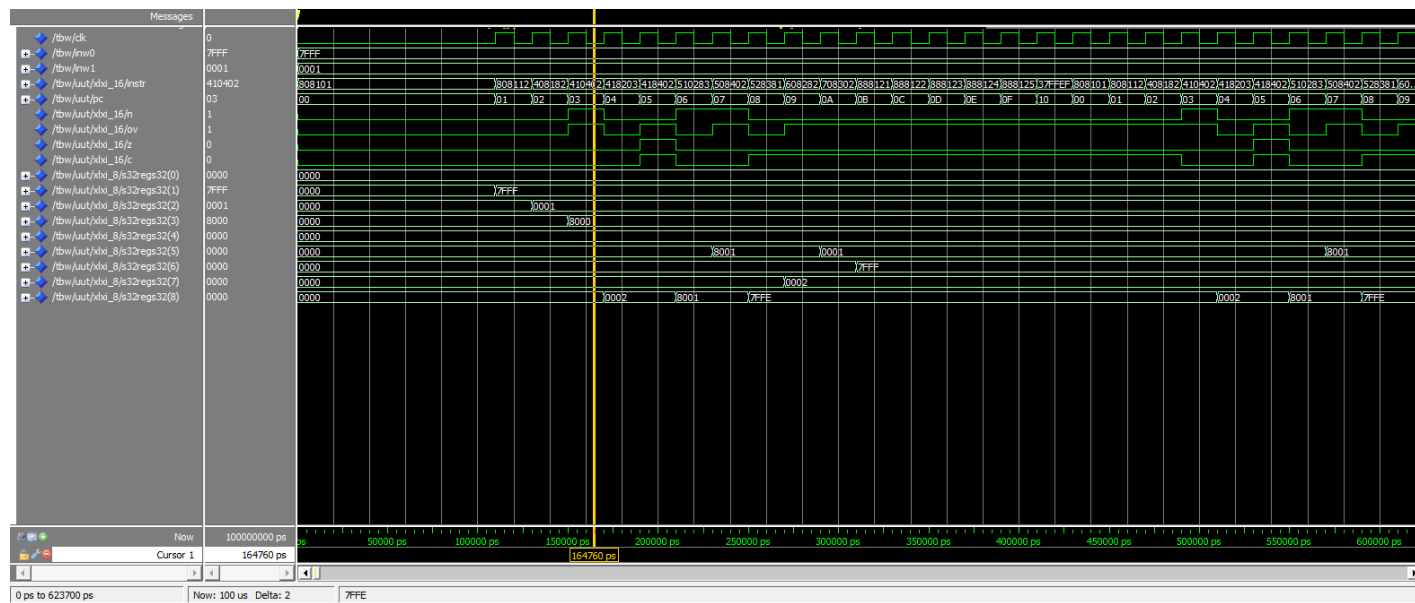
9. SECVENTA FLAG N:



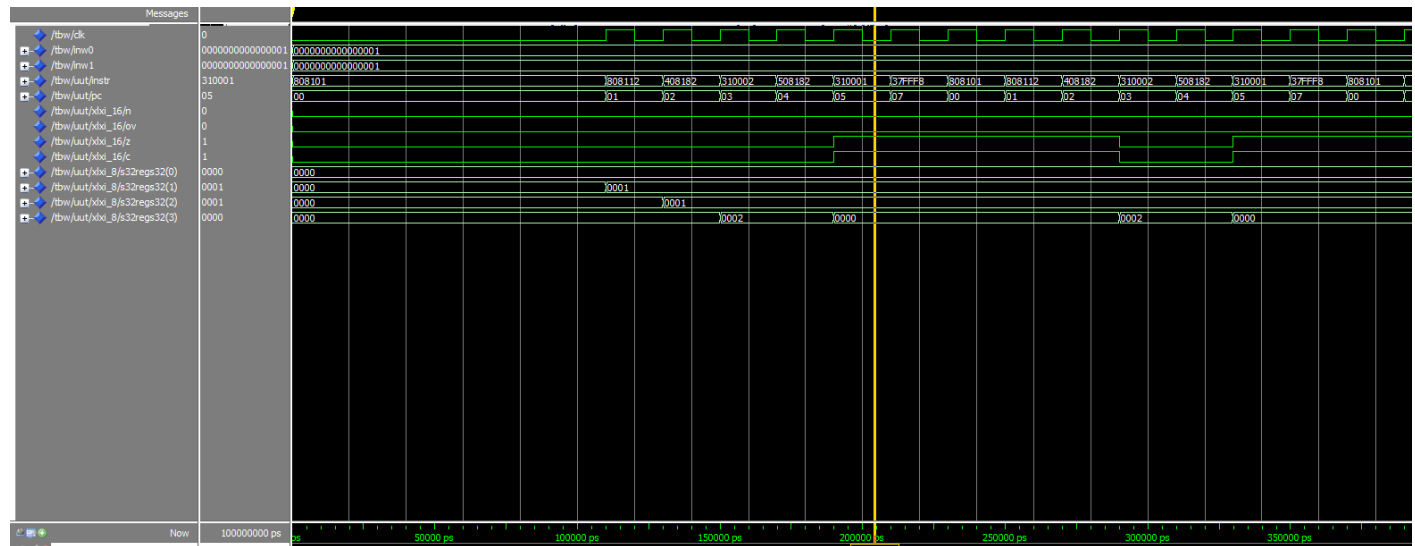
10. SECVENTA FLAG Z:



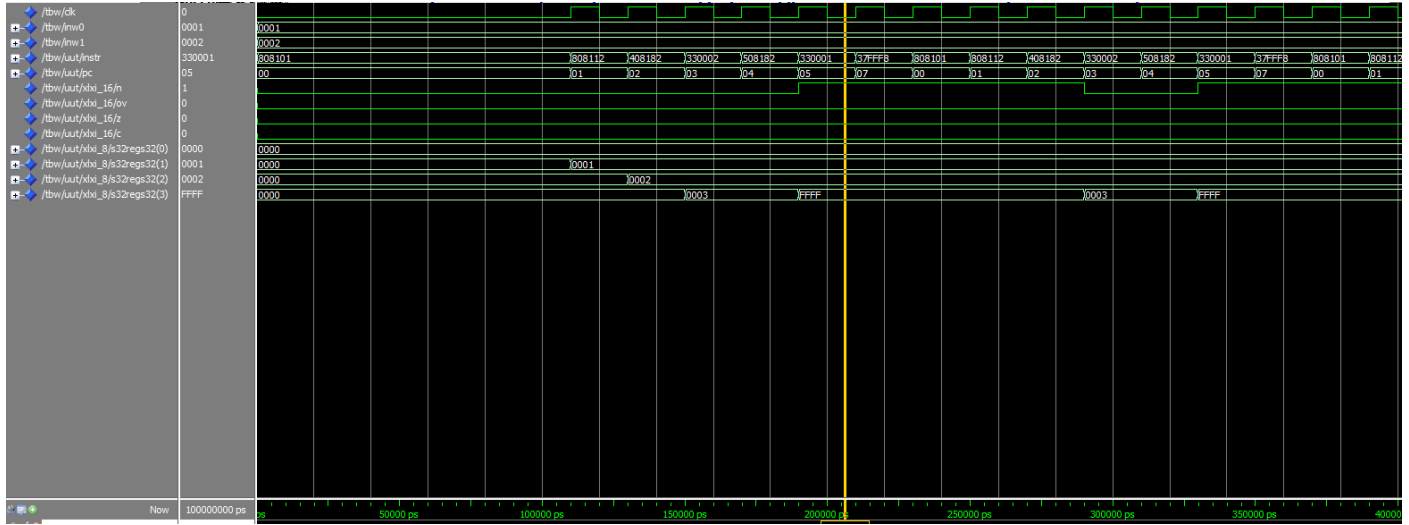
11. SECVENTA FLAG OV:



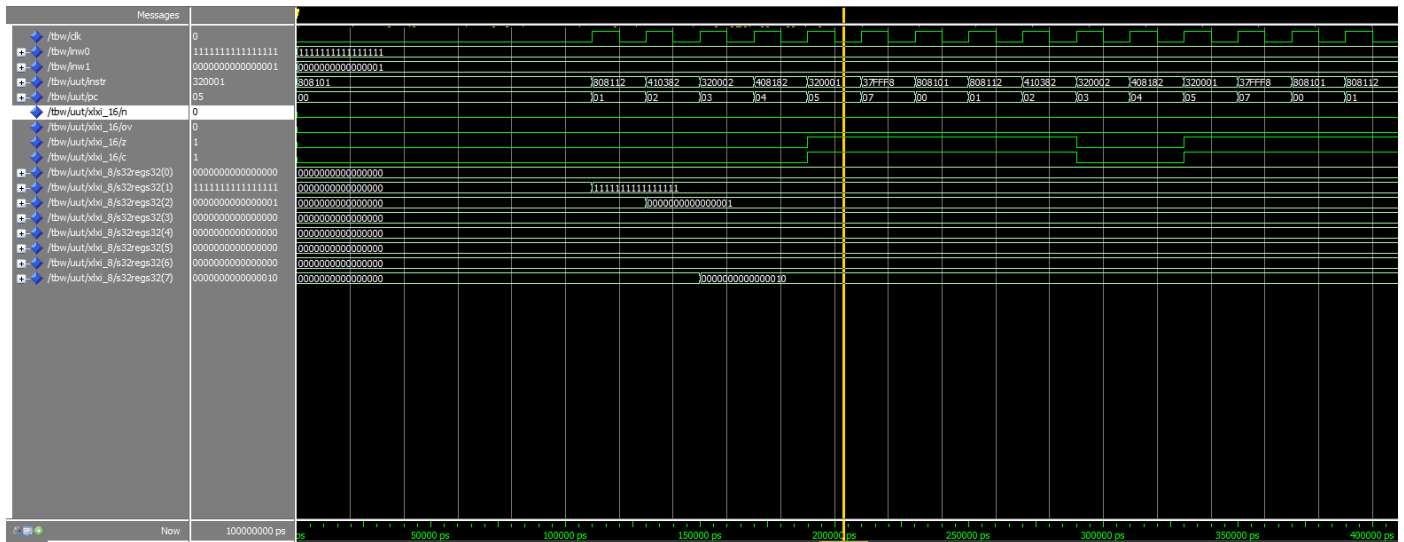
12. TEST BRA C:



13. TEST BRA N:



14.TEST BRA Z:



15. TEST BRA OV:

